

DAY 1 - PYTHON FUNDAMENTALS

Interview Preparation Question Bank | Important Questions Marked with ★

Scope	Questions	Important
Day 1 only - Python setup, Colab, variables, data types and operators	106	48

How to use this file: First answer every ★ question without looking at the answer. Then review the answer, write one example of your own, and repeat the questions aloud during interview revision.

A. Python Fundamentals

★ IMPORTANT What is Python?

Answer: Python is a high-level, general-purpose programming language known for readable syntax and a large ecosystem.

★ IMPORTANT Why is Python popular in Data Science and AI?

Answer: Because of its readable syntax, strong ecosystem, community support, and extensive data-science and AI tooling.

What are the major characteristics of Python?

Answer: High-level, dynamically typed, object-oriented, broadly interpreted in common usage, portable, readable, and extensible.

Is Python case-sensitive?

Answer: Yes. Python treats identifiers such as **name**, **Name**, and **NAME** as different names.

What does a high-level language mean?

Answer: It abstracts away many low-level machine details and provides syntax closer to human problem solving.

★ IMPORTANT Is Python compiled or interpreted?

Answer: Python is generally classified as interpreted. In CPython, source code is compiled to bytecode and then executed by the Python virtual machine.

What is CPython?

Answer: CPython is the standard and most widely used implementation of Python, written primarily in C.

What is bytecode in Python?

Answer: Bytecode is an intermediate representation of Python code that is executed by the Python virtual machine.

What is an interpreter?

Answer: An interpreter is software that executes code through a language runtime.

Why is Python considered portable?

Answer: Python programs can generally run across operating systems when a compatible Python environment and dependencies are available.

B. Python Environment and Google Colab

★ IMPORTANT What is Google Colab?

Answer: Google Colab is a browser-based notebook environment for writing and executing Python code.

Why is Google Colab useful for Data Science?

Answer: It provides an interactive environment with minimal setup and can provide access to accelerated computing resources depending on availability.

What is a Jupyter notebook?

Answer: An interactive document that combines executable code, output, text, equations, and visualizations.

What is the difference between a Python script and a notebook?

Answer: A script is primarily a code file executed as a program, while a notebook supports interactive cells, outputs, and explanatory content.

Why are notebooks popular in Data Science?

Answer: They make experimentation, visualization, documentation, and iterative analysis convenient.

What is a Python environment?

Answer: A configured or isolated setup containing a Python interpreter and project dependencies.

Why are virtual environments useful?

Answer: They isolate project dependencies and reduce package or version conflicts between projects.

What is a module?

Answer: A Python file containing reusable code such as functions, classes, or variables.

What is a package?

Answer: A collection of Python modules organized for reuse.

What happens when you execute a Python cell in Colab?

Answer: The code is sent to the connected runtime, executed by Python, and the output is returned to the notebook.

C. Variables and Typing

★ IMPORTANT What is a variable in Python?

Answer: A variable is a name that refers to an object.

★ IMPORTANT Does Python require explicit variable declaration?

Answer: No. You normally assign a value directly to a variable.

★ IMPORTANT What is dynamic typing?

Answer: The type associated with an object is determined at runtime, and a variable name can refer to objects of different types at different times.

★ IMPORTANT Is dynamic typing the same as weak typing?

Answer: No. Dynamic typing concerns when type information is determined; strong or weak typing concerns how strictly different types are handled.

★ IMPORTANT Is Python strongly typed?

Answer: Python is generally considered strongly typed because it does not freely perform arbitrary conversions between unrelated types.

What does this code demonstrate? `x = 10; x = "Python"`

Answer: The same variable name can refer to objects of different types at different times, illustrating dynamic typing.

How do you check the type of an object?

Answer: Use `type(x)`.

What is an identifier?

Answer: A name used to identify variables, functions, classes, and other objects.

What are the basic rules for Python identifiers?

Answer: They may contain letters, digits, and underscores; cannot begin with a digit; and cannot be reserved keywords.

Can a Python variable name contain spaces?

Answer: No. Use names such as `first_name` instead.

Can a variable name be a Python keyword?

Answer: No. Reserved keywords cannot normally be used as identifiers.

What naming convention is commonly used for Python variables?

Answer: `snake_case`, for example `customer_name`.

D. Python Data Types

★ IMPORTANT What is a data type?

Answer: A data type describes the kind of value an object represents and influences the operations applicable to it.

★ IMPORTANT What are common built-in Python data types?

Answer: `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `set`, `dict`, and `NoneType`.

What is an integer?

Answer: A whole-number type, such as `10`.

What is a float?

Answer: A floating-point numeric type, such as `10.5`.

What is a Boolean?

Answer: A logical type with the values True and False.

What is a string?

Answer: A sequence of text characters, such as "Python".

What is a complex number?

Answer: A number with real and imaginary components, for example $2 + 3j$.

★ IMPORTANT What is None?

Answer: A special singleton object representing the absence of a value.

What is the type of None?

Answer: NoneType.

★ IMPORTANT What is a mutable object?

Answer: An object whose contents can be changed after creation.

★ IMPORTANT What is an immutable object?

Answer: An object whose state cannot be changed after creation.

★ IMPORTANT Which common Python types are mutable?

Answer: list, dict, and set.

★ IMPORTANT Which common Python types are immutable?

Answer: int, float, bool, str, and tuple.

Why does string immutability matter?

Answer: An existing string object cannot be modified in place; operations that appear to change a string create or refer to another string object.

★ IMPORTANT Why is mutability important?

Answer: It affects how objects behave when modified, shared, or passed between parts of a program.

E. Numeric Types and Conversion

What is the difference between int and float?

Answer: int represents integers; float represents floating-point numbers.

Does Python support very large integers?

Answer: Yes. Python integers support arbitrary precision, subject to available memory.

★ IMPORTANT Why can floating-point calculations produce surprising results?

Answer: Because many decimal fractions cannot be represented exactly in binary floating-point.

How do you convert a string to an integer?

Answer: Use `int("100")`.

How do you convert an integer to a float?

Answer: Use `float(10)`.

How do you convert a value to a string?

Answer: Use `str(value)`.

★ IMPORTANT What is type conversion?

Answer: Changing a value from one data type to another.

What is explicit type conversion?

Answer: A conversion performed explicitly by the programmer, such as `int("10")`.

What is implicit type conversion?

Answer: A compatible conversion Python performs automatically in some operations, such as combining an integer with a float.

What happens with `int("10.5")`?

Answer: It raises a `ValueError` because the string is not a valid integer literal.

F. Operators - Arithmetic

★ **IMPORTANT** What are Python's basic arithmetic operators?

Answer: +, -, *, /, //, %, and **.

★ **IMPORTANT** What does / do?

Answer: Division. In Python 3, it produces a floating-point result.

★ **IMPORTANT** What does // do?

Answer: Floor division: the result is rounded down toward negative infinity.

★ **IMPORTANT** What does % do?

Answer: It returns the remainder from division.

What does ** do?

Answer: Exponentiation, for example `2 ** 3` gives 8.

★ **IMPORTANT** What is `7 // 2`?

Answer: 3.

★ **IMPORTANT** What is `7 % 2`?

Answer: 1.

What is `2 ** 4`?

Answer: 16.

★ **IMPORTANT** Why is `-10 // 3` equal to -4 rather than -3?

Answer: Because floor division floors toward negative infinity.

G. Comparison, Equality, Identity

★ **IMPORTANT** What are comparison operators?

Answer: ==, !=, >, <, >=, and <=.

★ **IMPORTANT** What does == mean?

Answer: Equality comparison: it checks whether two objects are equal according to their equality semantics.

★ **IMPORTANT** What is the difference between = and ==?

Answer: = performs assignment; == performs equality comparison.

★ **IMPORTANT** What are identity operators?

Answer: is and is not.

★ **IMPORTANT** What does is check?

Answer: Whether two references refer to the same object.

★ **IMPORTANT** What is the difference between == and is?

Answer: == checks equality; is checks object identity.

Why can two lists be == but not is?

Answer: Two separate list objects can contain equal values while still being different objects.

★ **IMPORTANT** Which operator should normally be used to compare values?

Answer: ==.

★ **IMPORTANT** What is a common use of is?

Answer: Checking singleton objects such as `x is None`.

H. Logical Operators

★ **IMPORTANT** What are Python's logical operators?

Answer: and, or, and not.

★ **IMPORTANT** What does and mean?

Answer: The combined condition is true when both required conditions are true.

★ **IMPORTANT** What does or mean?

Answer: The combined condition is true when at least one condition is true.

★ **IMPORTANT** What does not do?

Answer: It reverses the truth value of a Boolean expression.

What is the result of `10 > 5` and `10 < 20`?

Answer: True.

What is the result of `10 < 5` or `10 < 20`?

Answer: True.

What is the result of `not True`?

Answer: False.

I. Membership and Bitwise Operators

★ **IMPORTANT** What are Python's membership operators?

Answer: `in` and `not in`.

What does `in` do?

Answer: Checks whether an item is present in a container or iterable.

Give a membership example.

Answer: `3 in [1, 2, 3]` evaluates to True.

What does `not in` do?

Answer: Checks whether an item is absent.

What are Python's bitwise operators?

Answer: `&`, `|`, `^`, `~`, `<<`, and `>>`.

What does `&` mean?

Answer: Bitwise AND.

What does `|` mean?

Answer: Bitwise OR.

What does `^` mean?

Answer: Bitwise XOR.

What does `~` mean?

Answer: Bitwise NOT.

What do `<<` and `>>` mean?

Answer: Left shift and right shift.

J. Truthiness and Operator Precedence

★ **IMPORTANT** What are truthy and falsy values?

Answer: Python evaluates objects in Boolean contexts. Some values are treated as false and many others as true.

★ **IMPORTANT** Give common falsy values.

Answer: False, None, numeric zero, empty strings, and empty containers such as `[]`, `{}`, and `set()`.

Is an empty list truthy or falsy?

Answer: Falsy.

Is a non-empty list truthy or falsy?

Answer: Truthy.

★ **IMPORTANT** Is None the same as False?

Answer: No. They are different objects with different meanings, although None is falsy in Boolean contexts.

★ **IMPORTANT** What is operator precedence?

Answer: The rules that determine the order in which operators are evaluated.

What is the result of `2 + 3 * 4`?

Answer: 14, because multiplication has higher precedence than addition.

How can you explicitly control evaluation order?

Answer: Use parentheses.

K. High-Value Interview Traps

★ **IMPORTANT** What is the difference between a variable and an object?

Answer: A variable is a name/reference; an object is the entity that has a type and value/state.

★ **IMPORTANT** Can two variables refer to the same object?

Answer: Yes. For example, `b = a` makes `b` refer to the same object as `a`.

What is aliasing?

Answer: When multiple names refer to the same object.

★ **IMPORTANT** Why is aliasing important with mutable objects?

Answer: A modification through one reference can be visible through another reference to the same object.

★ **IMPORTANT** Why should you understand `is` instead of just memorizing it?

Answer: Because identity becomes important when reasoning about references, mutability, and object behavior.

★ **IMPORTANT** What is the core difference between dynamic typing and strong typing?

Answer: Dynamic typing concerns when type information is determined; strong typing concerns restrictions on operations between different types.

DAY 1 MASTER CHECK

1. Explain Python in 30 seconds.
2. Explain dynamic typing without confusing it with weak typing.
3. Explain variable vs object.
4. Explain mutable vs immutable with examples.
5. Explain `=` vs `==` and `==` vs `is`.
6. Explain `/` vs `//` and what `%` does.
7. Explain logical, membership, and identity operators.
8. Explain `None` and truthy/falsy values.
9. Explain why Google Colab is useful for Data Science.

Source alignment: This question bank is intentionally limited to Day 1 of the 120-day plan: Python setup + fundamentals, specifically installation/Colab, variables, types and operators. The plan instructs you to code every example rather than treating lectures as passive input.